

Argon2

1. Overview

Argon2 is a **password-hashing function** and the winner of the **Password Hashing Competition (PHC)** in **2015**, a multi-year open competition specifically run to identify the best modern successor to earlier password-hashing designs like bcrypt and scrypt.

Argon2 is the algorithm most current security guidance — including the **OWASP Password Storage Cheat Sheet** — recommends as the **default choice for new systems**.

Argon2 at a Glance

Property	Detail
Type	Dedicated, memory-hard password-hashing function
Selected via	Password Hashing Competition, 2015
Variants	Argon2d, Argon2i, Argon2id
Tunable parameters	Memory cost, time cost (iterations), degree of parallelism

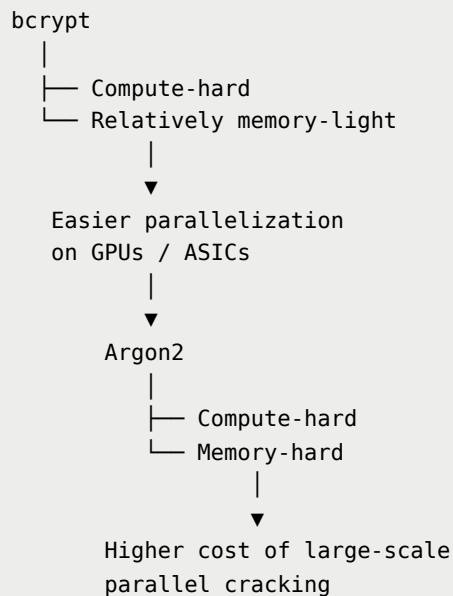
2. Why Memory-Hardness Matters

bcrypt (see its dedicated guide) is **compute-hard but relatively memory-light**, meaning an attacker with access to many cheap, parallel compute units — GPUs, and especially custom ASIC hardware — can still crack it faster than a single machine's raw compute-per-second figure would suggest.

Argon2 was explicitly designed to close this gap by requiring a configurable, deliberately large amount of **memory per single hashing attempt**.

Since building specialized hardware with large amounts of fast memory per parallel unit is significantly more expensive and difficult to scale than building compute-only ASICs, memory-hardness meaningfully raises the real-world cost of large-scale parallel cracking attacks.

Core Concept



3. The Three Argon2 Variants

Variant	Memory Access Pattern	Best Suited For
Argon2d	Data-dependent	Maximizes resistance to GPU-based cracking attacks. The memory access pattern depends on the input, making certain time-memory trade-off attacks harder — but this data-dependence is theoretically more exposed to side-channel (timing/cache) attacks.
Argon2i	Data-independent	Designed specifically to resist side-channel attacks because memory access patterns do not depend on secret input. It is slightly weaker against pure GPU-cracking resistance than Argon2d as a result.
Argon2id	Hybrid of both	Combines Argon2i's initial passes, resisting side-channel leakage, with Argon2d's later passes, resisting GPU cracking. It is the currently recommended default for password storage, balancing both threat models.

4. Tunable Parameters

Unlike bcrypt's single cost factor, Argon2 exposes **three independent parameters**, allowing much finer-grained tuning to a specific server's available

hardware and legitimate performance budget.

Parameter	Controls
Memory cost (m)	Amount of memory, in KB, required per hash computation
Time cost (t)	Number of iterations/passes over the memory
Parallelism (p)	Degree of parallel lanes used during computation

Parameter Selection

Recommended baseline parameters are periodically updated by security guidance bodies as hardware capabilities evolve.

A common current-generation starting point — subject to adjustment for a given server's actual capacity and latency budget — uses a **memory cost in the tens of megabytes range**, combined with a **small number of iterations** and **limited parallelism**.

Important: Parameter selection should be tuned for the actual server environment rather than copied blindly, because authentication performance and concurrent load vary between systems.

5. Built-In Salting and Output Format

Like bcrypt, Argon2 embeds a **unique, random salt automatically** as part of the algorithm's standard operation.

This is a structural property of correct usage, not something left to manual implementation.

A typical Argon2 hash string is **self-describing**, encoding:

- The variant
- The version
- All three tunable parameters
- The salt
- The resulting hash

This means a stored value carries everything needed to verify it later, even if the server's default parameters change over time.

Example Argon2 Hash

```
$argon2id$v=19$m=65536,t=3,p=4$<salt>$<hash>
```

Components

Component	Meaning
<code>argon2id</code>	Argon2 variant
<code>v=19</code>	Version
<code>m=65536</code>	Memory cost
<code>t=3</code>	Time cost / iterations
<code>p=4</code>	Degree of parallelism
<code><salt></code>	Unique random salt
<code><hash></code>	Resulting password hash

6. Why Argon2 Is the Current Recommended Default

Argon2 is recommended for several important reasons:

- Deliberately resists both **GPU/ASIC-parallelized brute-force attacks** through memory-hardness and, in the `id` variant, **side-channel leakage** simultaneously.
- Offers meaningfully more granular tuning than bcrypt's single exponential cost factor, allowing operators to balance security against legitimate authentication latency more precisely for their specific hardware.
- Emerged from an **open, rigorously reviewed public competition** specifically focused on password hashing, rather than being adapted from a cipher, as bcrypt was, or a general-purpose KDF, as PBKDF2 was — giving it a design purpose-built from first principles for this exact problem.

7. Practical Considerations

Capacity Planning

Argon2 requires more careful capacity planning than bcrypt.

Setting the memory cost too high across many concurrent authentication requests can itself become a **denial-of-service risk** on under-provisioned servers.

Therefore:

Parameter selection should be tested against **realistic concurrent load**, not just single-request latency.

Library Support

Library support has matured significantly and is now broadly available across major languages and frameworks.

However, it remains slightly less universally available than bcrypt in some older or niche ecosystems.

Periodic Parameter Review

As with any password-hashing scheme, parameters should be periodically reviewed and increased as hardware capability grows.

Existing password hashes can be **transparently re-hashed at the next login** under updated parameters where a migration is needed.

8. Typical Use Cases

Argon2 is commonly used for:

- **New application development** where password storage is required — the default recommended starting point per current OWASP guidance.
 - **Systems migrating away from legacy fast hashes**, such as MD5 or SHA-family hashes.
 - **Systems migrating from bcrypt** where enhanced GPU/ASIC resistance is specifically desired.
 - **Key-derivation contexts** more broadly, since Argon2 can also be used as a general-purpose password-based key derivation function beyond pure authentication use cases.
-

9. Summary

Argon2, and specifically its **Argon2id** variant, represents the current state of the art in password-hashing function design.

It combines:

1. **Deliberate memory-hardness**
2. **Resistance to side-channel leakage**
3. **Fine-grained parameter tuning**

These capabilities give it important advantages over earlier algorithms such as bcrypt.

Argon2id is the **recommended default for new systems**, while bcrypt remains an acceptable, well-tested alternative where Argon2 library support is not yet available.

Quick Reference

Feature	Argon2
Purpose	Password hashing
Memory-hard	✓ Yes
Variants	Argon2d, Argon2i, Argon2id
Recommended variant	Argon2id
Memory cost	Tunable
Time cost	Tunable
Parallelism	Tunable
Salt	Built in
Modern default	✓ Yes
Alternative	bcrypt where Argon2 support is unavailable